

AFRICAN LEADERSHIP UNIVERSITY

Introduction to Blockchain Development

Design and Implementation of a Blockchain-Based Library Book Lending Tracker

Technical Report

Student:	Aman Abraha Kasa
Course:	Introduction to Blockchain Development
Institution:	African Leadership University
Assignment:	Individual Assignment 1
Date:	September 2026
GitHub:	Github-Repo[FA:1]
Demo Video:	Demo-Vid(Youtube)

Table of Contents

1 Introduction.....	3
1.1 Background.....	3
1.2 Project Objective.....	3
1.3 Scope.....	3
1.4 Functional Requirements.....	3
2 System Architecture and Design Diagram.....	4
3 Blockchain Implementation.....	5
3.1 Block Structure.....	5
3.2 Genesis Block.....	6
3.3 Block Creation and Linking.....	6
4 Registry Management.....	7
4.1 Book and Member Registries.....	8
4.2 Registry Validation.....	8
5 Security Mechanisms.....	8
5.1 SHA-256 Hashing and Hash Linking.....	9
5.2 Digital Signatures and Key Management.....	9
5.3 Security Limitations.....	10
6 Data Persistence.....	10
7 Core Operations, Validation and Tamper Detection.....	11
7.1 Borrow, Return and View Records.....	11
7.2 Chain Validation.....	12
7.3 Tamper Demonstration.....	13
8 Error Handling and Data Validation.....	13
9 Testing and Results.....	14
10 Screenshots of Application Execution.....	15
11 Challenges Encountered and Solutions.....	18
11.1 Variable-length signatures in a fixed-size field.....	18
11.2 Choosing a persistence format.....	18
11.3 Showing tamper detection without weakening security.....	18
11.4 POSIX functions under strict C11.....	19
11.5 Unsafe string copy found during review.....	19
12 Discussion.....	19
13 Conclusion.....	19
14 References.....	20
Appendix A — Selected Code.....	21
Appendix B — Implementation Traceability.....	22

1 Introduction

1.1 Background

A library lending log records who borrowed which book and when it came back. In most libraries this log is a database table or a paper register, and whoever controls it can change it afterwards: a lost book can quietly be marked "returned", or a borrower can dispute an entry, and nothing in the log shows which version is authentic. An append-only ledger fixes this by linking each record to the one before it with a cryptographic hash and signing it with a private key. Editing a past record then breaks the hash, the link to the next record or the signature. The goal is not to stop anyone editing the file, which is impossible for a local file, but to make every edit **detectable** the next time the ledger is checked.

1.2 Project Objective

The project implements a command-line **Library Lending Chain** in C. Every borrow and return becomes a block in a linked-list blockchain. Each block is hashed with SHA-256, linked to the previous block through `previous_hash`, and signed with an ECDSA (NIST P-256) key using OpenSSL 3. Book and member IDs are checked against file-based registries before any block is created. The chain is saved to a text file after every change and re-verified every time the program starts. The source code is in `individual-assignment_FA-1/` of the GitHub repository `aman-kasa/blockchain-coursework`, split into six modules: `main.c`, `registry.c`, `blockchain.c`, `crypto.c`, `persistence.c` and `cli.c`.

1.3 Scope

In scope: loading `books.txt` and `members.txt`; a genesis block; signed BORROWED and RETURNED blocks; rejecting invalid IDs, double borrows and invalid returns; full chain validation at start-up, on demand and before every write; saving and reloading the chain from a file; and a persistent ECDSA key pair.

Out of scope: networking, peer nodes, consensus and mining, since this is a single process on one machine; user accounts, login and role-based access control (section 5.3); and automatic OVERDUE records, because the brief defines no due-date rule for them.

1.4 Functional Requirements

Table 1 maps each requirement in the brief to the code that implements it. The status column reflects what was verified by reading the code and running the program, not what the project's own documentation claims.

Table 1. *Assignment requirements mapped to the implementation*

Requirement (brief)	Implementation	File → function	Status
Blockchain of lending records	Linked list of Block structs (head, tail, length)	<code>blockchain.h</code> → Block, Blockchain	Met
Book / member registries from file	Loaded into <code>Book[256]</code> / <code>Member[256]</code> at start-up	<code>registry.c</code> → <code>registry_load()</code>	Met
Error if a registry file is missing or empty	Specific ERROR: message, then FATAL, exit code 1	<code>registry.c</code> ; <code>main.c</code> → <code>main()</code>	Met

Requirement (brief)	Implementation	File → function	Status
Reject unknown book / member IDs	ERROR: Book or Member not found	registry_find_book(), registry_find_member()	Met
Genesis: index 0, 64-zero previous_hash	Created on first run, hashed and signed	blockchain_create_genesis()	Met
SHA-256 links blocks	EVP_Digest over 8 fields incl. previous_hash	crypto_sha256_hex(), seal_block()	Met (note A)
Digital signatures	ECDSA P-256 over each block's hash	crypto_sign_hash(), crypto_verify_hash()	Met
Borrow / Return rules	ID, loan-state and chain-integrity checks before writing	blockchain_borrow(), blockchain_return()	Met
Validate chain (hash + link)	Both checks, plus index order and signature	blockchain_validate()	Met
View records incl. signature validity	All fields, hashes and a VALID/INVALID status per block	cli.c → print_records()	Met (note B)
Tamper detection; persistence; CLI	Text file data/chain.txt; 9 commands	persistence.c; cli.c → cli_run()	Met
Action OVERDUE	Fits in action[10] but is never created	—	Not implemented
Objective: user authentication and access control	One installation-wide signing key; no login or roles	crypto.c	Partial (5.3)

Note A — The brief describes `hash` as a hash of "all the above fields" and says to "sign it, compute its hash". The code does the reverse: it hashes the eight data fields (everything except `signature` and `hash`) and then signs that hash. The signature therefore still covers all the block's data.

Note B — `view records` checks each signature against the block's *stored* hash, so a block whose data was edited but whose stored hash was not changed still shows `signature=VALID` there. `validate chain` recomputes the hash and does catch it (section 7).

2 System Architecture and Design Diagram

Each of the six modules owns one concern and exposes it through a header file: `main.c` runs the start-up sequence, `registry.c` handles the registries, `blockchain.c` holds the chain rules and validation, `crypto.c` wraps every OpenSSL call, `persistence.c` owns the file format, and `cli.c` runs the command loop. Figure 1 is the system design diagram. It shows the registry path on the left, the borrow/return write path in the centre, and key handling, storage and verification on the right.

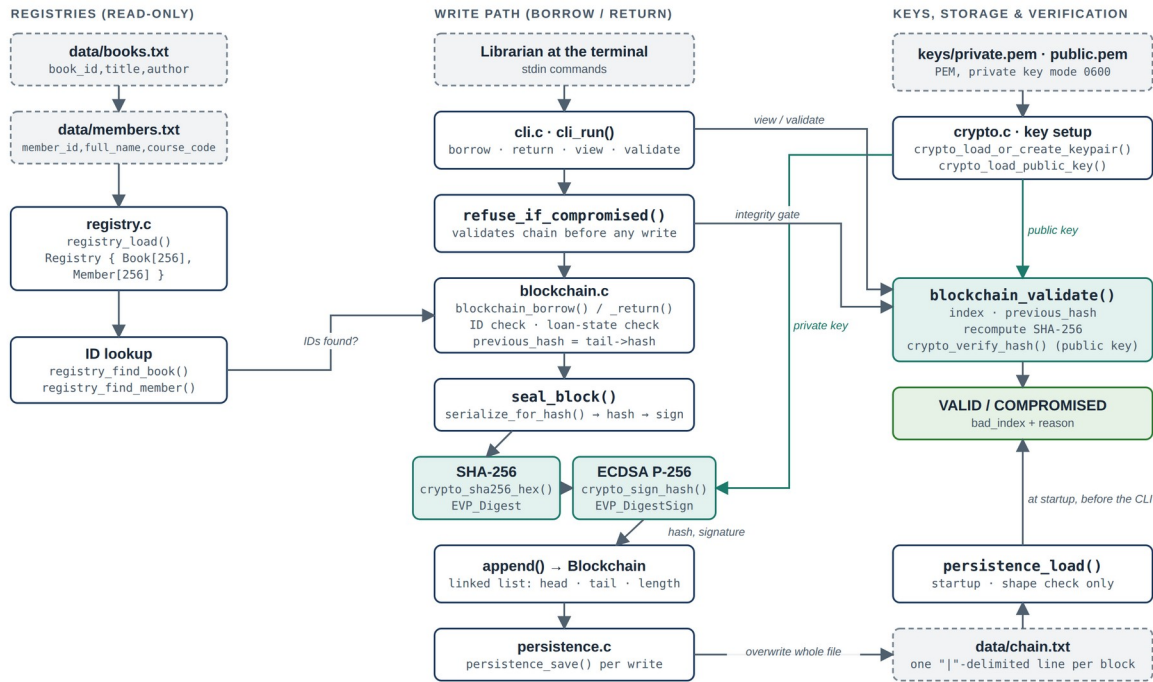


Figure 1. System design diagram: registries, lending (write) path, cryptography, storage and verification

Data flow. `registry_load()` reads both files once into a read-only Registry. A borrow or return passes through `refuse_if_compromised()`, which runs `blockchain_validate()` and refuses to write if the chain is already invalid. The business rules in `blockchain.c` then check the IDs and loan state and fill in the new block. `seal_block()` hashes and signs it, `append()` links it to the tail, and `persistence_save()` rewrites `data/chain.txt`. At start-up, `persistence_load()` rebuilds the chain and `blockchain_validate()` checks it, using the public key, before the first command is accepted.

3 Blockchain Implementation

3.1 Block Structure

`struct Block` in `blockchain.h` has the ten fields from the brief. Their sizes come from named constants (`BOOK_ID_LEN 20`, `BOOK_TITLE_LEN 80`, `MEMBER_NAME_LEN 50`, `ACTION_LEN 10`, `HASH_HEX_LEN 65`, `MAX_SIGNATURE_LEN 72`). Two extra fields exist only in memory: `next` for the linked list, and `sig_len` for the actual signature length (Challenge 11.1).

Table 2. Fields of struct Block

Field	Type	Purpose and source
index	int	Position in the chain (0 = genesis); set to chain->length when created
timestamp	time_t	Creation time, from time(NULL)
book_id, book_title	char[20], char[80]	Copied from the registry on borrow, and from the matching BORROWED block on return
member_id, member_name	char[20], char[50]	Same as above (the brief spells it Member_name; the code uses member_name)

Field	Type	Purpose and source
action	char[10]	GENESIS, BORROWED or RETURNED
previous_hash	char[65]	Hex SHA-256 hash of the previous block (64 zeros for genesis)
signature	unsigned char[72]	DER-encoded ECDSA signature over hash
hash	char[65]	Hex SHA-256 hash of the eight fields above
sig_len, next	size_t, Block *	In memory only: signature length and list link; neither is saved nor hashed

3.2 Genesis Block

On the first run, `persistence_load()` returns `PERSIST_NOT_FOUND` because there is no chain file yet. `main()` then prints `No existing chain found at 'data/chain.txt'` – creating genesis block. and calls `blockchain_create_genesis()`. That function refuses to run on a non-empty chain. It sets `index = 0`, the current timestamp and `action = "GENESIS"`, leaves the book and member fields empty, and fills `previous_hash` with 64 ASCII zeros using `blockchain_zero_hash()`. The genesis block is then sealed (hashed and signed) like any other block, appended, and saved straight away. Validation expects block 0's `previous_hash` to be the zero string, so a modified genesis block is rejected at index 0.

3.3 Block Creation and Linking

Figure 2 shows the exact order of checks for a borrow (`cli.c` → `handle_borrow()`, `blockchain.c` → `blockchain_borrow()`). The block is only allocated once every check has passed. Every block is sealed by one helper, which **hashes first and then signs the hash**:

```
src/blockchain.c – seal_block()
static int seal_block(Block *b, EVP_PKEY *pkey) {
    char buf[512];
    serialize_for_hash(b, buf, sizeof(buf));
    crypto_sha256_hex((const unsigned char *)buf, strlen(buf), b->hash);
    if (b->hash[0] == '\0') return 0;          /* hashing failed */
    if (!crypto_sign_hash(pkey, b->hash, b->signature,
                          sizeof(b->signature), &b->sig_len)) return 0;
    return 1;
}
```

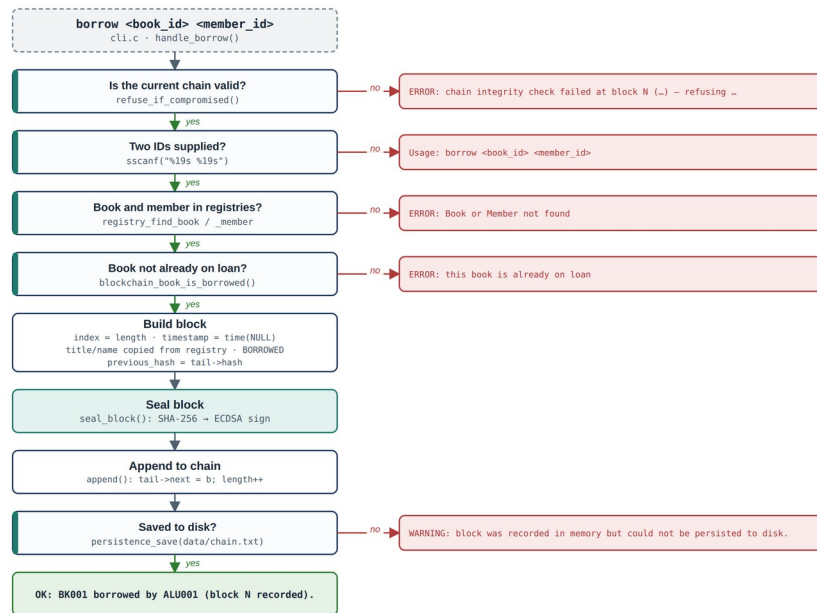


Figure 2. Borrow flow, showing the exact message printed on each failure

Linking. Before a new block is hashed, its `previous_hash` is copied from the current tail's hash. Because `previous_hash` is one of the hashed fields, each block's hash depends on the hash of the block before it, and so on back to the genesis block. Figure 3 shows a real three-block chain produced by the program.

Values from a sample run (hashes shortened). Red arrows: block N's hash is copied into block N+1's `previous_hash` when N+1 is created.



Figure 3. Block structure and chain linkage (values from a sample run, hashes shortened)

If block N is edited on disk, its recomputed hash no longer matches its stored hash. If the attacker also replaces the stored hash, block $N+1$'s `previous_hash` no longer matches. If they go on to rewrite every later hash, every one of those blocks fails its ECDSA check unless they also hold the private key.

4 Registry Management

`registry_load()` is called first in `main()` with `data/books.txt` and `data/members.txt`. Nothing else runs until both files have loaded. The Book (`book_id[20]`, `title[80]`, `author[50]`) and Member (`member_id[20]`, `full_name[50]`, `course_code[10]`) structs are identical to the brief. The repository ships the brief's sample data: BK001–BK003 and ALU001–ALU003.

4.1 Book and Member Registries

Both files use the same parsing code. Each line is read with `fgets()` into a 256-byte buffer, trailing `\r` and `\n` characters are removed, and blank lines are skipped. `split_fields()` splits the line on commas into at most three fields: `book_id`, `title`, `author` or `member_id`, `full_name`, `course_code`. A line without exactly three fields, or with a field too long for its buffer, is skipped with a `WARNING` and the other lines still load. Accepted fields are copied with bounded `snprintf()`, up to 256 entries per registry. The member file is only read after at least one valid book has loaded. `registry_find_book()` and `registry_find_member()` search linearly with an exact, case-sensitive `strcmp` and return `NULL` if nothing matches.

4.2 Registry Validation

Table 3. Registry validation outcomes (output captured from the compiled program)

Situation	Program output (exact)	Outcome
books.txt / members.txt missing	ERROR: Book registry file 'data/books.txt' not found or could not be opened. FATAL: could not load registries. Exiting.	Exit code 1
File empty / no valid lines	ERROR: Member registry 'data/members.txt' is empty or contains no valid records. + FATAL	Exit code 1
Malformed or oversized line	WARNING: malformed line in ... / WARNING: field too long in ...	Line skipped
Unknown book or member ID	ERROR: Book or Member not found	No block created

`return` takes only a `book_id`, so it checks only the book registry. The member is copied from the matching `BORROWED` block.

5 Security Mechanisms

All cryptography uses the **OpenSSL 3** `libcrypto` EVP interface (linked with `-lcrypto`), and every OpenSSL call is kept in `crypto.c`. No deprecated low-level functions such as `SHA256_Init` are used.

Table 4. Cryptographic primitives

Purpose	Algorithm	OpenSSL API	Wrapper
Block hash	SHA-256	<code>EVP_Digest(..., EVP_sha256())</code>	<code>crypto_sha256_hex()</code>
Key generation	EC, NIST P-256	<code>EVP_EC_gen("P-256")</code>	<code>generate_and_save_keypair()</code>
Key storage / load	PEM files	<code>PEM_write_PrivateKey</code> , <code>PEM_write_PUBKEY</code> , <code>PEM_read_*</code>	<code>crypto_load_or_create_keypair()</code> , <code>crypto_load_public_key()</code>
Sign / verify	ECDSA with SHA-256	<code>EVP_DigestSign</code> , <code>EVP_DigestVerify</code>	<code>crypto_sign_hash()</code> , <code>crypto_verify_hash()</code>

5.1 SHA-256 Hashing and Hash Linking

`serialize_for_hash()` joins eight fields, in a fixed order and separated by `|`, into one text buffer. Only signature and hash itself are left out. For block 1 of the sample run the hashed text is `1|1790449159|BK001|Things Fall Apart|ALU001|John Doe|BORROWED|c3e1b739...1c4b`.

```
src/blockchain.c – serialize_for_hash()
sprintf(buf, buf_len, "%d|%ld|%s|%s|%s|%s|%s|%s",
        b->index, (long)b->timestamp, b->book_id, b->book_title,
        b->member_id, b->member_name, b->action, b->previous_hash);
```

`crypto_sha256_hex()` passes this buffer to `EVP_Digest` once, when the block is sealed, and stores the 32-byte result as 64 lowercase hex characters in `hash`. If hashing fails it stores an empty string, which `seal_block()` treats as an error. During validation the same two functions recompute the hash of every loaded block and compare it with the stored value. A mismatch means one of the eight fields changed. `previous_hash` is the only thing that ties one block to the next: block 0 holds 64 zeros, and every later block holds the hash of the block before it (Figure 3).

5.2 Digital Signatures and Key Management

Keys. `main()` calls `crypto_load_or_create_keypair("keys/private.pem", "keys/public.pem")`. If both files exist, the private key is loaded. Otherwise a new P-256 key pair is generated with `EVP_EC_gen` and written as PEM files. The private-key file is set to owner-only access with `chmod(..., S_IRUSR | S_IWUSR)` (mode 0600) **before** any key material is written to it. A test run produced `-rw----- private.pem` and `-rw-r--r-- public.pem`. The public key is then loaded again, separately, with `crypto_load_public_key()`, so signing uses `priv_key` and all verification uses only the key stored in `public.pem`. `.gitignore` excludes `/keys/*.pem`, and no key material is embedded in the source or printed.

Signing. Inside `seal_block()`, straight after hashing, the block's 64-character hex hash string is signed with `EVP_DigestSign` using SHA-256, giving a DER signature of 70–72 bytes. A first call with a NULL buffer asks OpenSSL for the signature length, and anything longer than the 72-byte field is refused:

```
src/crypto.c – crypto_sign_hash() (return-value checks omitted)
EVP_DigestSignInit(ctx, NULL, EVP_sha256(), NULL, pkey);
EVP_DigestSign(ctx, NULL, &needed_len, (const unsigned char *)hash_hex, strlen(hash_hex));
if (needed_len > sig_out_cap) { /* "signature ... does not fit" */ goto done; }
EVP_DigestSign(ctx, sig_out, &sig_len, (const unsigned char *)hash_hex, strlen(hash_hex));
```

Verification. `crypto_verify_hash()` treats only an `EVP_DigestVerify` result of exactly 1 as valid (`valid = (rc == 1)`), so both a mismatch and an internal error count as invalid. A failed verification makes `blockchain_validate()` report ECDSA signature is invalid for this block's hash. The automated tests show the signature check works independently of the hash check: a signature with one hex digit changed is caught as a signature failure, and so is a chain checked against the wrong public key.

What a signature proves. A valid signature shows that the block's data was sealed by whoever held `keys/private.pem` at the time, which in practice means this installation of the program. It does not identify an individual librarian.

5.3 Security Limitations

- **One locally stored key.** The private key is an unencrypted PEM file on the same machine as the chain, protected only by file permissions. Anyone who can read it can rewrite and re-sign the whole chain so that it still validates.
- **No user authentication or access control.** There is no login, no per-user key and no role check. Anyone who can run the program can record transactions.
- **Deleting the newest blocks is not detected.** Blocks only link backwards, and nothing outside the file records the latest hash or the chain length. Removing the last line of `data/chain.txt` was tested: the program reported `Loaded chain: 3 block(s), integrity verified`. Deleting the file entirely makes the program start a new chain.
- **Local, not distributed.** There are no other nodes, no replicated copies and no consensus. The chain is tamper-evident, not tamper-proof. Saves are also not atomic: the file is rewritten in place with `fopen("w")`.

A production system would need per-user keys held in secure hardware, access rules, replicated or externally anchored chain heads (to detect deleted blocks), and atomic writes.

6 Data Persistence

The chain is stored in `data/chain.txt`, a plain-text file with one line per block and ten fields separated by `|`. No binary `chain.bin` file is used. The field order is `index|timestamp|book_id|book_title|member_id|member_name|action|previous_hash|signature_hex|hash`:

`data/chain.txt` from a test run (hex values shortened)

```
0|1790449159|0000|GENESIS|0000...0000|3046022100acb9...54345|c3e1b739...1c4b
1|1790449159|BK001|Things Fall Apart|ALU001|John Doe|BORROWED|c3e1b739...1c4b|30460221...40bb|
3c779512...9ca5
2|1790449159|BK001|Things Fall Apart|ALU001|John Doe|RETURNED|3c779512...9ca5|30450220...3b37|
89fafa0c...7409
```

Why text instead of binary. Writing raw structs to disk would depend on the compiler's padding, byte order and `time_t` size, and would save a meaningless `next` pointer. A text file avoids all of that and can be edited in any text editor, which is exactly what the tamper demonstration needs. Signature bytes are stored as hex.

Saving. `persistence_save()` opens the file in "w" mode and writes one `fprintf` line per block. It runs after the genesis block is created and after every successful `borrow` or `return`, so the file is always up to date even if the program is killed. If the file cannot be written, the program prints `WARNING: block was recorded in memory but could not be persisted to disk`.

Loading. `persistence_load()` rebuilds one block per line. `parse_line()` checks that each line has exactly 10 fields, that `index` and `timestamp` parse as integers (using `strtol` with `errno` and end-pointer checks), that every field fits its buffer, and that the signature is valid hex. `sig_len` is rebuilt from the decoded signature length. If any line fails, the result is `PERSIST_CORRUPT` and `main()` exits. The loader deliberately does not check hashes or signatures. `main()` calls `blockchain_validate()` straight after loading, so all cryptographic checks happen in one place (Figure 4).

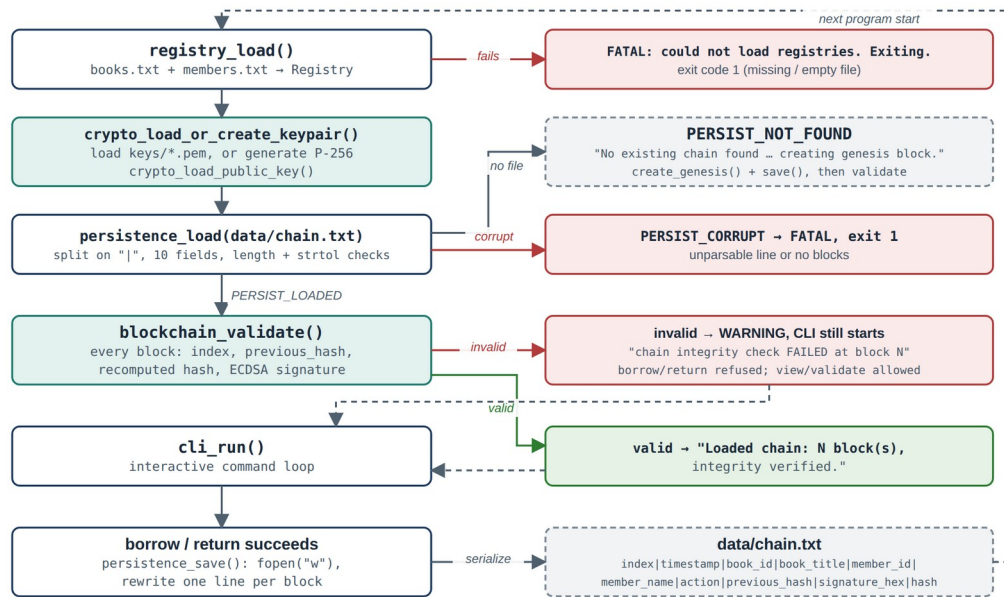


Figure 4. Start-up sequence and persistence lifecycle (main.c)

7 Core Operations, Validation and Tamper Detection

7.1 Borrow, Return and View Records

Borrow (borrow <book_id> <member_id>) follows Figure 2: integrity check, argument parsing with bounded `sscanf("%19s %19s")`, registry lookup, a loan-state check, block creation, sealing, append and save. It ends with OK: BK001 borrowed by ALU001 (block 1 recorded).

Return (return <book_id>). `find_last_for_book()` walks the whole chain, skips the genesis block, and keeps the **last** block with a matching `book_id`, which is the book's most recent lending event. If there is no such block, or its action is not `BORROWED`, the book is not on loan and the program prints ERROR: this book was never borrowed, or has already been returned. Otherwise a `RETURNED` block is created with the book and member details copied from that `BORROWED` block, then sealed, appended and saved. Loan status is never stored separately; it is always worked out from the chain.

View records (view records / view). `print_records()` prints every block with its action, book ID and title, member ID and name, the local time (`localtime_r` and `strftime("%Y-%m-%d %H:%M:%S")`), hash, previous hash, and signature=VALID or signature=INVALID from `crypto_verify_hash()`.

Table 5. CLI commands (cli.c → cli_run()); matched case-insensitively

Command	Handler	Result
borrow <book_id> <member_id>	handle_borrow()	OK: ... borrowed by ... (block N recorded). or an error
return <book_id>	handle_return()	OK: ... returned (block N recorded). or an error
view records / view	print_records()	All blocks with fields, hashes and signature status

Command	Handler	Result
validate chain / validate / chain status	print_status()	Chain status: N block(s), VALID or COMPROMISED + first bad index and reason
list books / list members	cli_run()	Registry contents
help, exit / quit	print_help(), loop exit	Command list; quit
anything else	—	Unrecognized command: '...'. Type 'help' for the command list.

7.2 Chain Validation

`blockchain_validate()` walks the chain once and returns { valid, bad_index, reason } for the **first** failure it finds. An empty chain is invalid. `expected_prev_hash` starts as 64 zeros and `expected_index` as 0. Each block is then checked in the order shown in Table 6.

Table 6. Checks applied to every block, in order (`blockchain.c` → `blockchain_validate()`)

#	Check	Comparison	Reason reported on failure
0	Index order	<code>b->index</code> vs <code>expected_index</code>	block index out of sequence
1	Link	<code>b->previous_hash</code> vs <code>expected_prev_hash</code> : 64 zeros for block 0, otherwise the previous block's hash	<code>previous_hash</code> does not match the prior block's hash
2	Hash integrity	Stored <code>b->hash</code> vs SHA-256 recomputed from <code>serialize_for_hash(b)</code>	stored hash does not match recomputed hash (block data was modified)
3	Signature	<code>crypto_verify_hash(pub_key, b->hash, b->signature, b->sig_len)</code>	ECDSA signature is invalid for this block's hash

```
src/blockchain.c – blockchain_validate() (abridged)
for (const Block *b = chain->head; b; b = b->next) {
    if (b->index != expected_index)                { /* fail: index */ }
    if (strcmp(b->previous_hash, expected_prev_hash) != 0) { /* fail: link */ }
    serialize_for_hash(b, buf, sizeof(buf));
    crypto_sha256_hex((const unsigned char *)buf, strlen(buf), recomputed);
    if (strcmp(recomputed, b->hash) != 0)           { /* fail: hash */ }
    if (!crypto_verify_hash(pub_key, b->hash, b->signature, b->sig_len))
                                                    { /* fail: sig */ }

    snprintf(expected_prev_hash, HASH_HEX_LEN, "%s", b->hash);
    expected_index++;
}
```

Check 1 compares against the previous block's *stored* hash. This is safe because that block has already passed Check 2. Validation runs in three places: at start-up, on request, and inside `refuse_if_compromised()` before every write.

7.3 Tamper Demonstration

The following was run against the compiled program in a separate test directory:

1. **Valid chain.** After borrow BK001 ALU001 and return BK001, validate chain printed Chain status: 3 block(s), VALID.
2. **Past block edited.** In data/chain.txt, block 1's title was changed from Things Fall Apart to Xhings Fall Apart, a single byte. The stored hash was left as it was.
3. **Hash no longer matches.** The stored hash 3c779512...9ca5 was computed from the original title, so the recomputed hash is different.
4. **Detected at start-up.** On restart the program printed WARNING: chain integrity check FAILED at block 1: stored hash does not match recomputed hash (block data was modified).
5. **Reported and blocked.** validate chain printed Chain status: 3 block(s), COMPROMISED and first problem at block index 1: ... A following borrow BK002 ALU002 was refused with ERROR: chain integrity check failed at block 1 (...) – refusing to add new records until this is resolved.

Figures 12 and 13 in section 10 show the same demonstration run on the author's machine. The hash values differ from the ones above because the blocks have different timestamps, but the program output is the same.

Refusing new writes matters. Without it, a new block would link to the tampered block's stored hash and be signed correctly, which would make the damaged history harder to spot. tests/run_tests.sh also automates tampering with the stored hash, with previous_hash, with a single byte, with a signature, and with the public key.

8 Error Handling and Data Validation

Errors fall into three levels. **Fatal at start-up** (exit code 1): the registries, keys or chain file cannot be used. **Flagged, not fatal:** the chain file parses but fails validation, so the CLI starts and view/validate work, but writes are refused. **Per command:** the program prints a message and keeps running.

Table 7. Error scenarios handled by the program

Error scenario	Detected in	Program response
Registry file missing or empty	registry.c → registry_load()	ERROR: ... then FATAL: could not load registries. Exiting.
Malformed or oversized registry line	registry.c → split_fields(), length checks	WARNING: ...; line skipped
Unknown book or member ID	blockchain_borrow() / blockchain_return()	ERROR: Book or Member not found
Book already on loan	blockchain_book_is_borrowed()	ERROR: this book is already on loan
Return of a book not on loan	find_last_for_book()	ERROR: this book was never borrowed, or has already been returned
Missing arguments / unknown command	cli.c → sscanf count, cli_run()	Usage: ... / Unrecognized command: ...

Error scenario	Detected in	Program response
Chain file unparsable or empty	<code>persistence.c</code> → <code>parse_line()</code> , <code>persistence_load()</code>	ERROR: chain file ... unparsable ... then FATAL, exit 1
Tampered chain (index, link, hash, signature)	<code>blockchain_validate()</code>	Start-up WARNING; COMPROMISED with block index and reason; writes refused
Key generation or load failure	<code>crypto_load_or_create_keypair()</code> , <code>crypto_load_public_key()</code>	ERROR: with the OpenSSL error string, then FATAL, exit 1
Hashing or signing failure; signature > 72 bytes	<code>seal_block()</code> , <code>crypto_sign_hash()</code>	Block freed; ERROR: internal cryptographic operation failed
Chain file cannot be written	<code>persistence_save()</code>	WARNING: block was recorded in memory but could not be persisted to disk.
Out of memory	<code>calloc</code> checks	Error returned; nothing appended

Defensive C. All copies into fixed-size fields use `snprintf`. User input is read with bounded `%19s` conversions, numbers are parsed with checked `strtol` rather than `atoi`, and hex decoding is bounds-checked. Every block and OpenSSL object is freed exactly once. The code compiles with `-std=c11 -Wall -Wextra -Wpedantic` with **zero warnings**.

9 Testing and Results

There are two sources of evidence. The first is `tests/run_tests.sh` (`make test`), which runs the compiled program in a fresh temporary directory for each test. The second is a manual session against the compiled program. Both were run on 26 September 2026 with gcc 13.3.0 and OpenSSL 3.0.13 on Linux, and the automated suite reported **44 passed, 0 failed**.

Table 8. Automated test suite (`tests/run_tests.sh`): 44/44 assertions passed

Group	Assertions	Coverage
Registry	11	Valid load; missing or empty books/members file; malformed line; oversized field
Borrow / Return	5 / 4	Valid borrow and return; invalid book or member; double borrow; never borrowed; already returned
Blockchain	9	Genesis created only once; 4-block chain; reload in a new process; titles and names from the registry; corrupted genesis; duplicate index
Cryptography	5	All signatures valid; corrupted signature; wrong public key
Tampering	7	Edited field, stored hash or <code>previous_hash</code> ; single-byte flip; write refused
Memory safety	3	Full session under AddressSanitizer/UBSan: no errors or leaks

Table 9. *Required demonstrations: manual session and automated results*

#	Test case	Input / action	Actual result (program output)	Status
T1	Registry loading	start; list books; list members	Books (3): ... Members (3):	PASS
T2	Missing registry	remove books.txt, start	ERROR: Book registry file ... not found ..., FATAL, exit 1	PASS
T3	Valid borrow	borrow BK001 ALU001	OK: BK001 borrowed by ALU001 (block 1 recorded).	PASS
T4	Invalid book / member ID	borrow ZZZ ALU001; borrow BK002 ALU999	ERROR: Book or Member not found (both)	PASS
T5	Double borrow	borrow BK001 ALU002	ERROR: this book is already on loan	PASS
T6	Return / invalid return	return BK001 (twice); return BK003	OK: BK001 returned (block 2 recorded)., then ... never borrowed, or has already been returned (×2)	PASS
T7	View records	view records	3 blocks, each signature=VALID	PASS
T8	Valid chain	validate chain	Chain status: 3 block(s), VALID	PASS
T9	Tampered chain	edit block 1 title; restart; validate chain	WARNING ... at start-up; COMPROMISED, first problem at block index 1	PASS
T10	Write on tampered chain	borrow BK002 ALU002	ERROR: chain integrity check failed ... refusing to add new records ...	PASS
T11	Persistence	exit and restart	Loaded chain: 2 block(s), integrity verified. (automated test)	PASS
T12	Delete newest block	remove the last line of chain.txt	Loaded chain: 3 block(s), integrity verified.: not detected	Gap (5.3)

10 Screenshots of Application Execution

Screenshots were taken after a clean build (make clean && make), with data/chain.txt and keys/*.pem removed first so the first-run behaviour is visible. Each figure caption names the requirement it demonstrates.

```

aman@Aman-PC:~/Documents/low-level-progn/blockchain/blockchain-coursework/individual-assignment_FA-1$ ./lending_tracker
No existing chain found at 'data/chain.txt' – creating genesis block.
Loaded chain: 1 block(s), integrity verified.
Library Lending Chain – type 'help' for commands.
> help
Commands:
borrow <book_id> <member_id>   Record a book as borrowed
return <book_id>                Record a book as returned
view records                    Show every lending record on the chain
validate chain                  Verify chain integrity (hashes, links, signatures)
list books                     Show the book registry
list members                   Show the member registry
chain status                    Show block count and validity at a glance
help                           Show this message
exit                           Quit
> list books
Books (3):
BK001   Things Fall Apart           Chinua Achebe
BK002   Americanah                  Chimamanda Ngozi Adichie
BK003   The River Between           Ngũgĩ wa Thiong'o
> list members
Members (3):
ALU001   John Doe                   BLK101
ALU002   Jane Smith                 BLK101
ALU003   Amara Diallo               BLK101
>

```

Figure 5. Program start-up: registries loaded and genesis block created

Shows a genesis block (index 0) created and verified on the first run, the command list from help, and both registries loaded into memory at start-up.

```

aman@Aman-PC:~/Documents/low-level-progn/blockchain/blockchain-coursework/individual-assignment_FA-1$ ls -l keys/
total 8
-rw----- 1 aman aman 241 Sep 26 21:53 private.pem
-rw-rw-r-- 1 aman aman 178 Sep 26 21:53 public.pem
aman@Aman-PC:~/Documents/low-level-progn/blockchain/blockchain-coursework/individual-assignment_FA-1$

```

Figure 6. Generated ECDSA key pair, with owner-only permissions on the private key

Shows that a persistent P-256 key pair is stored outside the source code and that private.pem is readable only by its owner (-rw-----).

```

aman@Aman-PC:~/Documents/low-level-progn/blockchain/blockchain-coursework/individual-assignment_FA-1$ mv data/books.txt data/books.bak
./lending_tracker
mv data/books.bak data/books.txt
ERROR: Book registry file 'data/books.txt' not found or could not be opened.
FATAL: could not load registries. Exiting.
aman@Aman-PC:~/Documents/low-level-progn/blockchain/blockchain-coursework/individual-assignment_FA-1$

```

Figure 7. Error when books.txt is missing

Shows the required error message and clean exit when a registry file cannot be found.

```

> borrow BK001 ALU001
OK: BK001 borrowed by ALU001 (block 1 recorded).
> borrow ZZZ ALU001
ERROR: Book or Member not found
> borrow BK002 ALU999
ERROR: Book or Member not found
> borrow BK001 ALU002
ERROR: this book is already on loan
> █

```

Figure 8. Valid borrow; rejected invalid book ID, invalid member ID and double borrow

Shows that book and member IDs are checked against the registries before any block is created and that a book cannot be borrowed twice.

```

> return BK001
OK: BK001 returned (block 2 recorded).
> return BK001
ERROR: this book was never borrowed, or has already been returned
> return BK003
ERROR: this book was never borrowed, or has already been returned
> █

```

Figure 9. Successful return and rejected invalid returns

Shows that a return is only recorded when the book's most recent record is BORROWED.

```

ERROR: this book was never borrowed, or has already been returned
> view records
Lending chain (3 blocks):
[0] GENESIS time=2026-09-26 21:56:43
    hash=655bb19c03d024bc33e3540106b6ec39e0746a8a2e2befed593c638f28d9e8ea
    prev=0000000000000000000000000000000000000000000000000000000000000000
    signature=VALID
[1] BORROWED book=BK001 (Things Fall Apart) member=ALU001 (John Doe) time=2026-09-26 22:00:31
    hash=4f990c38fd4832a0a52d79f8ce7937e2e4e21d427afd09b7bdb4f887603c9211
    prev=655bb19c03d024bc33e3540106b6ec39e0746a8a2e2befed593c638f28d9e8ea
    signature=VALID
[2] RETURNED book=BK001 (Things Fall Apart) member=ALU001 (John Doe) time=2026-09-26 22:01:49
    hash=bd81b69d6df082cab8bc10deb0f575e90ffd0cb51b9d0b348d466dca8b7a2976
    prev=4f990c38fd4832a0a52d79f8ce7937e2e4e21d427afd09b7bdb4f887603c9211
    signature=VALID
> █

```

Figure 10. Lending history with hashes, previous hashes and signature status

Shows all records with title, member name, action, timestamp and signature validity, and each block's prev equal to the previous block's hash.

```

signature=VALID
> validate chain
Chain status: 3 block(s), VALID
> exit
aman@Aman-PC:~/Documents/low-level-progn/blockchain/blockchain-coursework/individual-assignment_FA-1$ █

```

Figure 11. validate chain on an unmodified chain

Shows the chain passing all four validation checks before any tampering.

[illegible]

Figure 12. *A past block edited outside the program*

Shows a single-character change to a historical record made directly in the chain file.

```
aman@Aman-PC:~/Documents/low-level-progn/blockchain/blockchain-coursework/individual-assignment_FA-1$ ./lending_tracker
WARNING: chain integrity check FAILED at block 1: stored hash does not match recomputed hash (block data was modified)
The chain file may have been tampered with. Borrow/return are disabled until this is resolved;
use 'view records' and 'validate chain' to inspect the damage.
Library Lending Chain - type 'help' for commands.
> validate chain
Chain status: 3 block(s), COMPROMISED
  first problem at block index 1: stored hash does not match recomputed hash (block data was modified)
> borrow BK002 ALU002
ERROR: chain integrity check failed at block 1 (stored hash does not match recomputed hash (block data was modified)) - refusing to add new records until this is r
esolved.
> exit
```

Figure 13. Start-up warning, COMPROMISED status and refused write after tampering

Shows that the edit is detected, traced to block 1 with its reason, and that no new records are accepted until it is resolved.

11 Challenges Encountered and Solutions

These challenges are recorded in the repository: in code comments, the `Makefile`, `docs/ARCHITECTURE.md`, `docs/REQUIREMENTS.md` and `docs/REPORT.md`. The git history keeps only the final source files, so the "before" state is taken from that documentation.

11.1 Variable-length signatures in a fixed-size field

Challenge: The brief defines `unsigned char signature[72]` with no length field. **Cause:** DER-encoded P-256 ECDSA signatures vary between 70 and 72 bytes, and verification needs the exact length.

Solution: An in-memory-only field, `size_t sig_len`, was added, and `crypto_sign_hash()` checks the size before writing. On load, the length is rebuilt from the hex string by `crypto_hex_to_bytes()` (comment in `blockchain.h`; REQUIREMENTS.md ambiguity 4). **Result:** The brief's ten fields are unchanged, and 71- and 72-byte signatures both verify after a save and reload.

11.2 Choosing a persistence format

Challenge: The chain has to survive restarts and be easy to edit externally for the tamper demonstration. **Cause:** Writing raw structs to disk depends on padding, byte order and `time_t` size, saves a meaningless `next` pointer, and needs a hex editor to modify.

Solution: A `|`-delimited text format with hex signatures was chosen. The loader checks only the file's structure and leaves cryptographic checks to `blockchain_validate()` (ARCHITECTURE.md, "Persistence strategy"). **Result:** The file is portable and editable in any text editor, malformed files are rejected cleanly, and all tamper detection happens in one function.

11.3 Showing tamper detection without weakening security

Challenge: The program has to start on a tampered chain so `validate_chain` can show the failure, but it must not keep accepting records. **Cause:** According to `docs/REPORT.md` §18, an earlier version exited on any validation failure at start-up, which made a live demo

impossible. Simply starting anyway would let a correctly signed new block be added on top of the tampered history.

Solution: A failed start-up validation became a warning instead of a fatal error, and `refuse_if_compromised()` re-validates the chain before every write (comments in `main.c` and `cli.c`). **Result:** Tampering can be inspected, but no writes are accepted; the test "borrow is refused while the chain is compromised" passes.

11.4 POSIX functions under strict C11

Challenge: `localtime_r()` is not declared when compiling with `-std=c11`. **Cause:** In strict ISO C mode, glibc hides POSIX extensions unless a feature-test macro is defined.

Solution: `_D_POSIX_C_SOURCE=200809L` was added to `CFLAGS` in the Makefile. **Result:** The code builds with `-Wall -Wextra -Wpedantic` and produces no warnings.

11.5 Unsafe string copy found during review

Challenge: A `strcpy` was found in the validation loop, against the project's rule of using only bounded string functions. **Cause:** Both buffers were the same size, so the compiler gave no warning.

Solution: A `git grep` for unsafe functions found it, and it was replaced with `snprintf` (REQUIREMENTS.md R11.2). **Result:** The source contains no `strcpy`, `strcat` or `gets`, and the sanitizer test session is clean.

12 Discussion

The project demonstrates the core data structure behind a blockchain: an ordered, append-only list anchored by a genesis block, in which each record commits to the one before it through a SHA-256 hash, and in which loan status is always worked out from the chain's own history. Integrity is enforced, not just recorded: every hash is recomputed on every start-up and before every write, and the tests show that each of the four checks catches its own kind of tampering. ECDSA adds something hashing alone cannot provide, because anyone can recompute a SHA-256 hash but only the holder of the private key can produce a valid signature. The tests with a corrupted signature and with the wrong public key confirm that verification really distinguishes valid from invalid signatures.

The limits are just as real. With one node and one installation-wide key, and no consensus or replication, this is not comparable to Bitcoin or Ethereum. Anyone with the private key can rewrite history, and deleting the newest blocks goes unnoticed. What the project shows, working and tested, is the *tamper-evidence* property of a single ledger.

13 Conclusion

This project implemented a library lending tracker in C that records every borrow and return as a block in a linked-list blockchain. Book and member IDs from `books.txt` and `members.txt` are checked before any record is created. Each block is hashed with SHA-256 over its data and its predecessor's hash, then signed with a persistent ECDSA P-256 key through OpenSSL 3. The chain is saved to a text file after every change and fully re-validated at every start-up, checking index order, hash links, recomputed hashes and signatures, and naming the first block that fails. Editing a past record is detected and blocks further writes, as shown both manually and by 44 passing automated tests. The course objectives on blockchain structure, SHA-256 integrity, persistence and error handling are met. Authentication is achieved

through block signatures made with the system's key. User-level authentication and access control are not implemented, and are the main area for further work.

14 References

- [1] African Leadership University (2026). *Individual Assignment 1: Design and Implementation of a Blockchain-Based Library Book Lending Tracker*. Introduction to Blockchain Development, September 2026 Term.
- [2] National Institute of Standards and Technology (2015). *FIPS 180-4: Secure Hash Standard*. <https://doi.org/10.6028/NIST.FIPS.180-4>
- [3] National Institute of Standards and Technology (2023). *FIPS 186-5: Digital Signature Standard*. <https://doi.org/10.6028/NIST.FIPS.186-5>
- [4] National Institute of Standards and Technology (2023). *SP 800-186: Elliptic Curve Domain Parameters* (defines P-256). <https://doi.org/10.6028/NIST.SP.800-186>
- [5] OpenSSL Project. *OpenSSL 3.0 manual pages: EVP_DigestSignInit(3), EVP_DigestVerifyInit(3), EVP_DigestInit(3), EVP_PKEY-EC(7), PEM_read_bio_PrivateKey(3)*. <https://docs.openssl.org/3.0/>

Appendix A — Selected Code

The main text already shows `seal_block()` (3.3), `serialize_for_hash()` (5.1), `crypto_sign_hash()` (5.2) and the `blockchain_validate()` loop (7.2). This appendix adds block creation, persistence and the integrity check that runs before every write.

A.1 `src/blockchain.c` – `blockchain_borrow()`

```
LendResult blockchain_borrow(Blockchain *chain, const Registry *reg, EVP_PKEY *pkey,
                             const char *book_id, const char *member_id) {
    const Book *book = registry_find_book(reg, book_id);
    const Member *member = registry_find_member(reg, member_id);
    if (!book || !member) {
        return !book ? LEND_ERR_BOOK_NOT_FOUND : LEND_ERR_MEMBER_NOT_FOUND;
    }
    if (blockchain_book_is_borrowed(chain, book_id)) return LEND_ERR_ALREADY_BORROWED;

    Block *b = calloc(1, sizeof(Block));
    if (!b) return LEND_ERR_CRYPTO;
    b->index = (int)chain->length;
    b->timestamp = time(NULL);
    snprintf(b->book_id, BOOK_ID_LEN, "%s", book->book_id);
    snprintf(b->book_title, BOOK_TITLE_LEN, "%s", book->title);
    snprintf(b->member_id, MEMBER_ID_LEN, "%s", member->member_id);
    snprintf(b->member_name, MEMBER_NAME_LEN, "%s", member->full_name);
    snprintf(b->action, ACTION_LEN, "BORROWED");
    snprintf(b->previous_hash, HASH_HEX_LEN, "%s", chain->tail->hash);

    if (!seal_block(b, pkey)) { free(b); return LEND_ERR_CRYPTO; }
    append(chain, b);
    return LEND_OK;
}
```

A.2 `src/persistence.c` – `persistence_save()`

```
int persistence_save(const Blockchain *chain, const char *path) {
    FILE *f = fopen(path, "w");
    if (!f) { fprintf(stderr, "ERROR: could not open '%s' for writing the chain.\n", path);
    return 0; }
    for (const Block *b = chain->head; b; b = b->next) {
        char sig_hex[MAX_SIGNATURE_LEN * 2 + 1];
        crypto_bytes_to_hex(b->signature, b->sig_len, sig_hex, sizeof(sig_hex));
        fprintf(f, "%d|%d|%s|%s|%s|%s|%s|%s|\n", b->index, (long)b->timestamp,
                b->book_id, b->book_title, b->member_id, b->member_name,
                b->action, b->previous_hash, sig_hex, b->hash);
    }
    fclose(f);
    return 1;
}
```

A.3 src/cli.c – refuse_if_compromised()

```
static int refuse_if_compromised(const Blockchain *chain, EVP_PKEY *pub_key) {
    ChainValidation v = blockchain_validate(chain, pub_key);
    if (!v.valid) {
        printf("ERROR: chain integrity check failed at block %d (%s) – refusing to add new
records "
            "until this is resolved.\n", v.bad_index, v.reason);
        return 1;
    }
    return 0;
}
```

Appendix B — Implementation Traceability

Table 10. *Feature-to-code traceability*

Feature	File	Function / structure	Section
Block, chain	blockchain.h/.c	struct Block, Blockchain, append()	3.1
Genesis	blockchain.c	blockchain_create_genesis(), blockchain_zero_hash()	3.2
Registries	registry.h/.c	Book, Member, registry_load(), registry_find_*	4
Borrow / Return	cli.c, blockchain.c	handle_borrow/return(), blockchain_borrow/return(), find_last_for_book()	3.3, 7.1
SHA-256	blockchain.c, crypto.c	serialize_for_hash(), crypto_sha256_hex(), seal_block()	5.1
ECDSA and keys	crypto.c	crypto_load_or_create_keypair(), crypto_sign_hash(), crypto_verify_hash()	5.2
Validation	blockchain.c, cli.c	blockchain_validate(), refuse_if_compromised()	7.2, 7.3
Persistence	persistence.c	persistence_save(), persistence_load(), parse_line()	6
Start-up, CLI	main.c, cli.c	main(), cli_run(), print_records(), print_status()	2, 7.1
Build, tests	Makefile, tests/run_tests.sh	make, make test (44 assertions), make asan	9